and do this sampling over a period of time, you will end up with what is called *continuous profiling* (CP; e.g., what the eBPF-based Parca (https://www.parca.dev/) project offers. We will discuss the concept of CP, its use cases, and its tooling in detail in chapter 9.

- Distributed traces are used to propagate a context across a number of microservices along a request path in the context of a distributed system, such as a containerized microservice. After the request is completed, you can stitch together the separate service executions, often visualized in a waterfall view or a graph view showing the service invocations, often called *service map*.

NOTE The term *waterfall visualization* is neither new nor unique to distributed tracing. If you look at the in-browser developer tooling available nowadays, you will find that it uses a similar representation to show how long the browser spends handling events (http://mng.bz/o1Od), such as DOM elements handling, or networking-related actions, such as downloading style files or images.

Now, let's take a high-level look at distributed traces, but if you're interested in the details, you can skip ahead to chapter 8, where we will dive deep into the topic. In a nutshell, distributed tracing is all about capturing and visualizing how a request moves through your microservices, potentially across different machines and involving different protocols such as HTTP or gRPC. Two terms you will often come across in this context are

- *Trace*—Represents the end-to-end processing of the request as it moves through the participating microservices. A trace can have multiple spans.
- *Span*—Represents the execution in one particular service along the request path, with the root span as the first span in a trace and child spans as the subsequent (nested) ones.

But how does this work? When a request enters the system, a unique identifier (the trace ID) is generated. Then, depending on the protocol the services used to communicate, such as HTTP or gRPC, this trace ID along with some payload is sent on to the children. This is what we call *context propagation* across processes in a networked setup.

The distributed tracing solution abstracts away the technical details of how the propagation is implemented as well as takes care of stitching all the spans together and delivers them to a central place where you then can view, query, and analyze the traces.

Combining distributed traces with what logs offer yields some interesting insights:

- You can look up the active trace context, pinpointing interesting sources quickly.
- If you attach a log event to the span, you can directly jump from the waterfall/timing visualization to the relevant log message of a component. This